Prompt: Fix FlashDrop Camera Functionality (Android Kotlin + Jetpack Compose)

In my existing FlashDrop Android application, the Camera option is not working when I click the button to take a photo. Fix the actual problem and implement fully working camera functionality. Do not redesign the existing UI or remove any existing features.

1. Diagnose the existing problem

- Inspect the current camera button, click handler, Activity/Composable, ViewModel, and image-selection code.
- Find why clicking the Camera button does not open the camera.
- Check Android permissions, Activity Result APIs, camera launch logic, AndroidManifest.xml, FileProvider configuration if needed, and error handling.
- Fix the root cause instead of adding a dummy camera button.

2. Open the device camera

- When the user clicks the Camera icon or "Take Photo" button, open the phone's camera using a suitable Android camera integration.
- Use Kotlin and Jetpack Compose-compatible Activity Result APIs, such as `ActivityResultContracts.TakePicturePreview` for a simple preview or `ActivityResultContracts.TakePicture` for saving a full-resolution photo.
- Prefer saving the photo to an appropriate app-specific location when the image must be uploaded or used later.
- Handle devices that do not have an available camera or camera application.

3. Handle permissions correctly

- Request camera permission when required by the chosen camera implementation.
- Add the necessary AndroidManifest.xml permissions.
- Handle permission denial gracefully and explain how the user can enable permission in Android Settings.
- Do not repeatedly request permission if the user has permanently denied it.
- Avoid requesting unnecessary storage permissions.

4. Save and display the photo

- After the user takes a photo, return to FlashDrop and show the captured image in the existing image preview.
- Let the user retake the photo or remove it before submitting.
- Keep the image available during the current form session.
- If the user confirms the photo, save or upload it using the project's existing image-storage architecture.

5. Integrate with FlashDrop features

- Make the camera work wherever the existing application offers photo capture, especially in the Shopkeeper Add Product form.
- If profile photos or other existing forms already support camera capture, repair those flows too.

- Preserve the existing Gallery/Image Picker option, if available.
- Ensure that the captured product image is correctly associated with the selected product and shopkeeper account.

6. Error handling

- Handle camera cancellation, permission denial, missing camera apps, invalid image URIs, file creation failures, and upload errors.
- Show clear error messages instead of silently doing nothing.
- Add useful debug logs to identify failures during testing.
- Prevent duplicate launches from repeated button taps.

7. Test the implementation

- Clicking Camera opens the camera.
- Taking a photo returns the image to FlashDrop.
- Cancelling returns to the previous screen without crashing.
- Denying permission displays a helpful message.
- The image preview appears correctly and the saved image remains associated with the product after submission.
- Test on an actual Android phone as well as an emulator where possible.

Inspect the existing project files and modify the actual implementation. Do not create a separate demo screen, fake camera preview, or non-functional button. Preserve the current FlashDrop design, authentication, database, product management, and other features. At the end, explain the root cause, list the files changed, and report any Android permissions or configuration changes required.